🕸️ Graph Data Structure Notes

📚 What is a Graph?

A **graph** is a data structure that represents a collection of **nodes (vertices)** connected by **edges**. It's used to model relationships such as networks, maps, or connections between entities.

🔹 Basic Terminology

Term	Meaning
Vertex (Node)	An individual data point in the graph.
Edge	A connection between two vertices.
Degree	Number of edges incident to a vertex.
Path	A sequence of vertices connected by edges.
Cycle	A path that starts and ends at the same vertex.

🔸 Common Types of Graphs

1. Undirected Graph

- Edges have no direction (A — B means A is connected to B and vice versa).
- Example: Friendship network.

2. Directed Graph (Digraph)

- Edges have direction (A → B means from A to B).
- Example: Instagram "follows".

3. Weighted Graph

- Each edge has a weight or cost.
- Example: Road map with distances or time.

4. Unweighted Graph

- All edges are treated equally (no weights).

5. Cyclic Graph

- Contains at least one cycle.

6. Acyclic Graph

- No cycles (e.g., **DAG** – Directed Acyclic Graph).

7. Connected Graph

- Every vertex can reach every other vertex.

8. Disconnected Graph

- Some vertices are not reachable from others.

9. Complete Graph

- Every vertex is connected to every other vertex.

◈ Graph Representation Methods

1. Adjacency Matrix

A 2D array (V × V) where each cell `[i][j]` shows whether there is an edge between vertex `i` and `j`.

✓ Example

For vertices **A, B, C**:

	A	**B**	**C**
A	0	1	1
B	1	0	0
C	1	0	0

+ Pros

- Easy to implement.
- Quick to check if two nodes are connected (`O(1)` lookup).

− Cons

- Uses **O(V^2)** space, even for sparse graphs.
- Slower to iterate through neighbors.

---

2. Adjacency List

Each vertex stores a **list of its neighbors**.

✓ Example

```
A → [B, C]
B → [A]
C → [A]
```

+ Pros

- Efficient for **sparse graphs**.
- Takes **O(V + E)** space.

− Cons

- Checking if an edge exists takes **O(degree of vertex)** time.

---

3. Edge List

Simply store all edges as **pairs (or triplets)** of vertices.

✓ Example

For graph with edges (A–B), (A–C):

```
[(A, B), (A, C)]
```

For weighted graph:

```
[(A, B, 5), (A, C, 2)]
```

+ Pros

- Very simple representation.
- Best for algorithms that iterate over edges (e.g., Kruskal's MST).

− Cons

- Slow for adjacency checks.
- Not ideal for traversal (like BFS/DFS).

□ Summary Table

Representation	Space Complexity	Edge Lookup	Best For
Adjacency Matrix	$O(V^2)$	O(1)	Dense graphs
Adjacency List	O(V + E)	O(deg(V))	Sparse graphs
Edge List	O(E)	O(E)	Edge-based algorithms